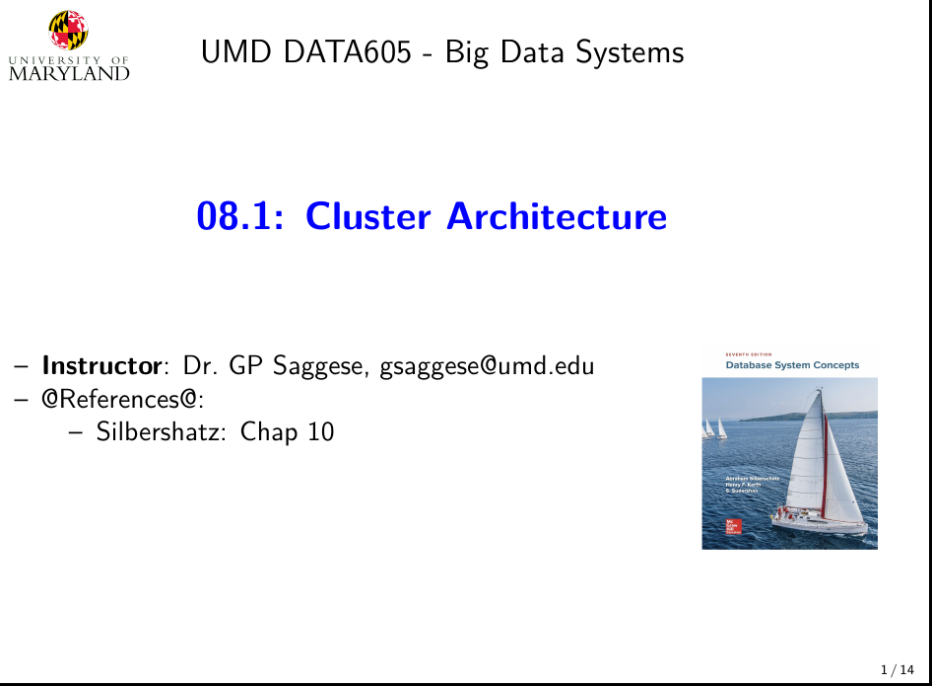
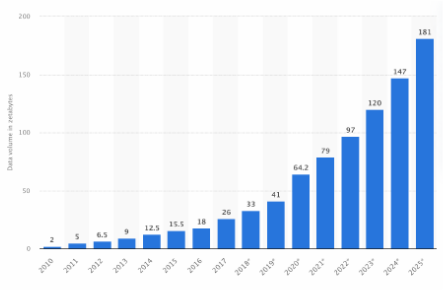
08.1: Cluster Architecture



2 / 14: Big Data: Storing and Computing

Big Data: Storing and Computing



Volume of data in the world

- **Big data needs 10k-100k machines**
- **Two problems**
  - Storing big data
  - Processing big data
- **Need to solve problems together and efficiently**
  - One slow phase slows entire system

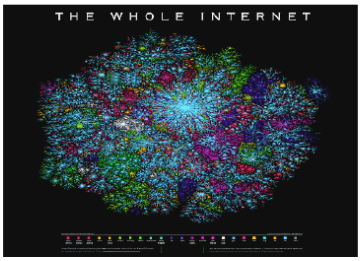
2 / 14

- **Big data needs 10k-100k machines**
  - As the volume of data grows exponentially, managing it requires a vast number of machines. This scale is necessary to handle the storage and processing demands efficiently.
  - The graph illustrates the rapid increase in data volume, highlighting the need for substantial computational resources.
- **Two problems**
  - *Storing big data*: With data volumes reaching zettabytes, traditional storage solutions are inadequate. New methods and technologies are needed to store this data efficiently and cost-effectively.
  - *Processing big data*: Beyond storage, the ability to process and analyze data quickly is crucial. This requires powerful computing resources and optimized algorithms.
- **Need to solve problems together and efficiently**
  - Both storage and processing must be addressed simultaneously. If one aspect lags, it can bottleneck the entire system, reducing overall efficiency.
  - Integrated solutions that optimize both storage and processing are essential to handle the growing data demands effectively.

3 / 14: Processing the Web: Example

Processing the Web: Example

- Web contains (in 2024):
  - 20+ billion pages
  - 5M TBs of content
- **Storing the web**
  - Need 1M 5TB hard drives
    - $100/HDD -> $100M in total
    - Not bad
  - Wait, one computer reads 300 MB/sec
    - 4,500 years to read web serially!
- **Processing the web**
  - Much larger time / cost needed!

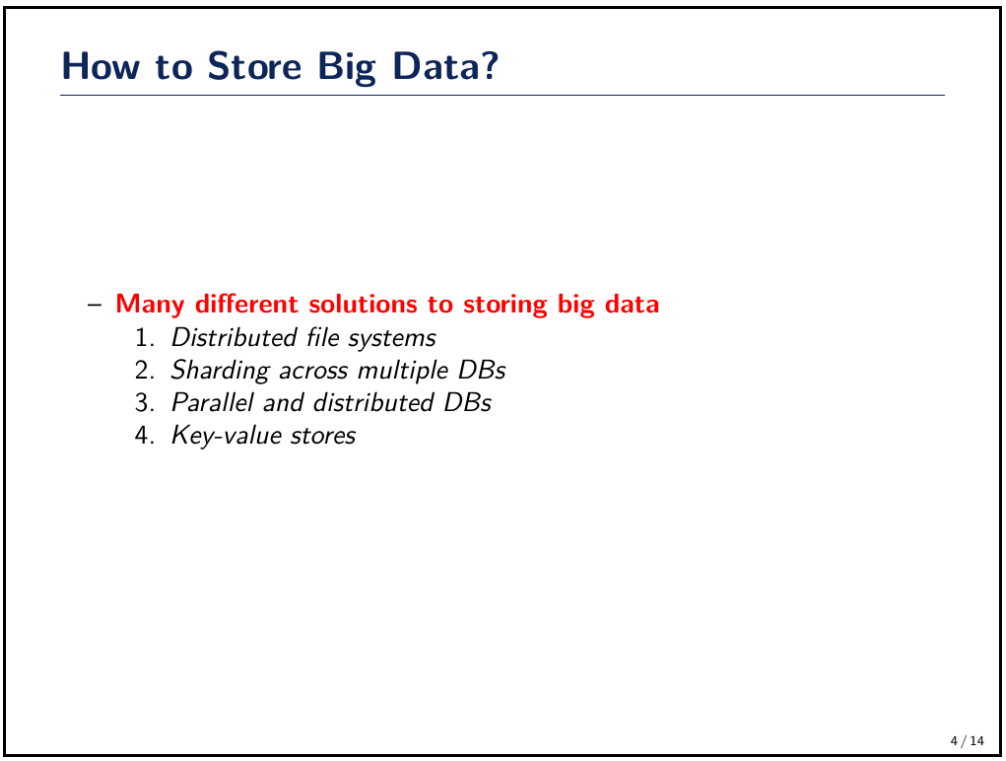


3 / 14

- **Web contains (in 2024):**
  - The web is vast, with over 20 billion pages. This highlights the immense scale of information available online.
  - The content amounts to 5 million terabytes (TB), illustrating the massive data volume that needs to be managed and processed.
- **Storing the web:**
  - To store the entire web, you would need 1 million hard drives, each with a capacity of 5TB. This gives a sense of the physical storage requirements.
    - At a cost of $100 per hard drive, the total expense would be $100 million, which is surprisingly affordable given the scale.
  - However, a single computer reading at 300 MB per second would take 4,500 years to read all this data sequentially. This emphasizes the impracticality of processing such data with traditional methods.
- **Processing the web:**
  - The time and cost required to process the web are significantly larger than just storing it. This underscores the challenges in handling and analyzing such vast amounts of data efficiently.

The image visually represents the complexity and interconnectedness of the internet, reinforcing the scale and intricacy of web data.

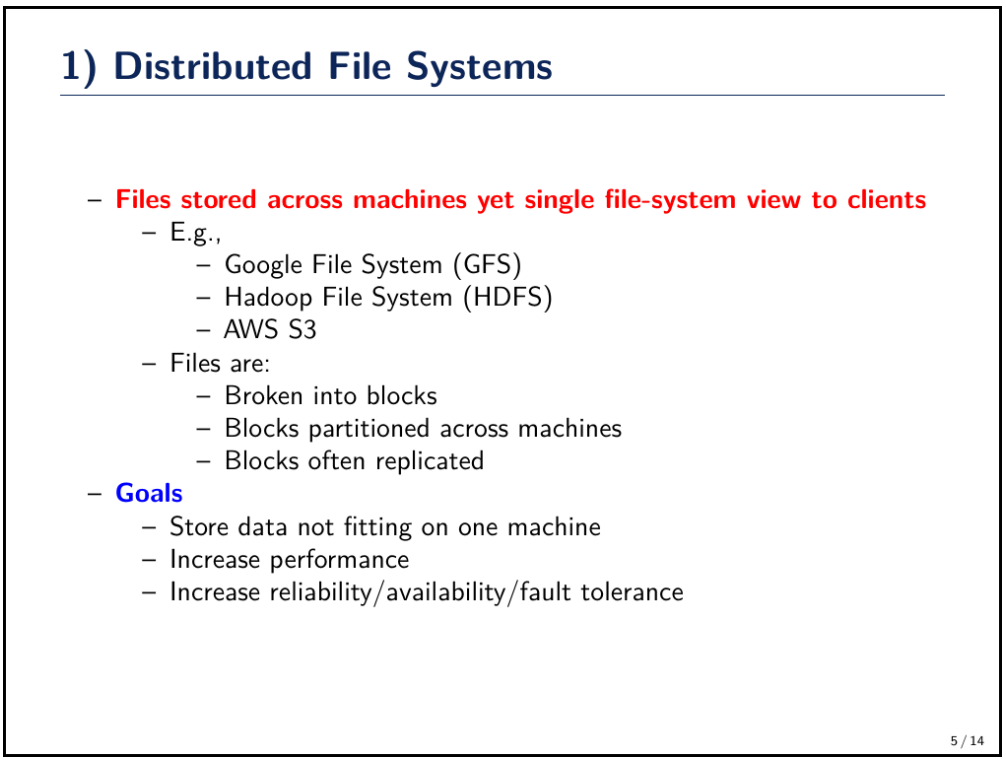
4 / 14: How to Store Big Data?



– **Many different solutions to storing big data**
  – When dealing with big data, traditional storage methods often fall short due to the sheer volume, velocity, and variety of data. Therefore, specialized solutions are necessary to efficiently store and manage big data.

1. ***Distributed file systems***
   – Distributed file systems, like Hadoop Distributed File System (HDFS), are designed to store large amounts of data across multiple machines. They ensure data is replicated across different nodes to provide fault tolerance and high availability. This means if one machine fails, the data is still accessible from another machine.
2. ***Sharding across multiple DBs***
   – Sharding involves splitting a database into smaller, more manageable pieces, called shards, which are distributed across multiple databases. This approach helps in scaling out databases horizontally, allowing them to handle more data and traffic by adding more servers.
3. ***Parallel and distributed DBs***
   – These databases, such as Google Bigtable or Amazon Redshift, are designed to process queries in parallel across multiple servers. This parallel processing capability allows them to handle large datasets efficiently and provide faster query responses.
4. ***Key-value stores***
   – Key-value stores, like Redis or Amazon DynamoDB, are a type of NoSQL database that store data as a collection of key-value pairs. They are highly scalable and can handle large volumes of data with low latency, making them suitable for applications requiring fast read and write operations.

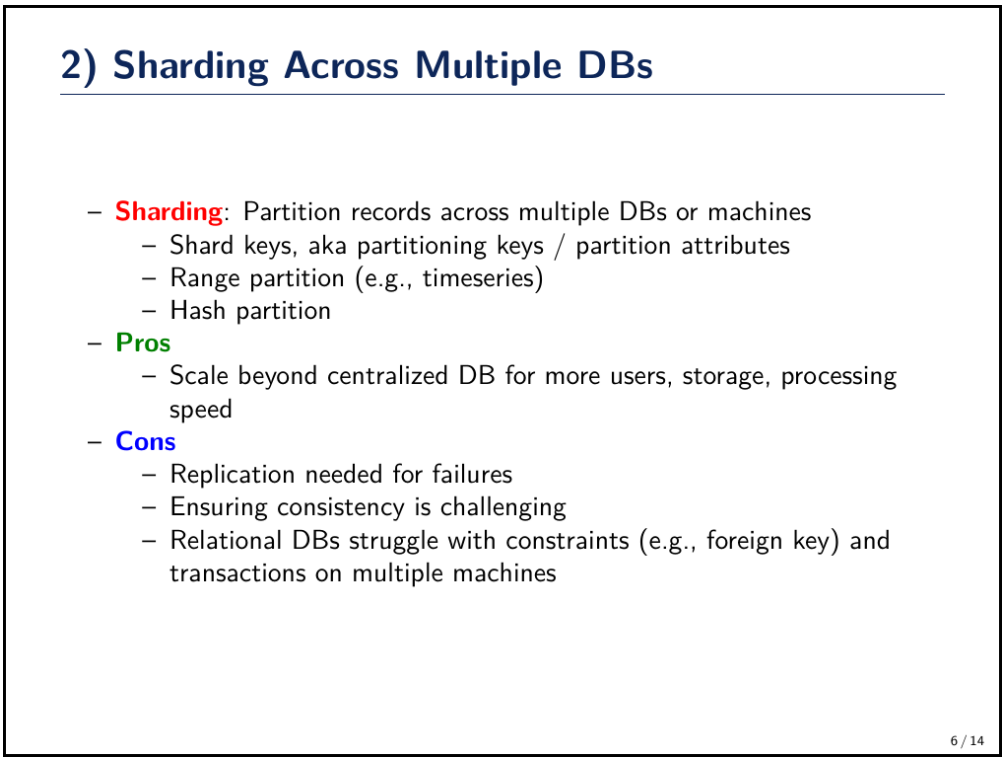
5 / 14: 1) Distributed File Systems



- **Distributed File Systems**
- *Files stored across machines yet single file-system view to clients*
  - Distributed file systems allow files to be stored across multiple machines, but they appear as a single file system to users. This means that even though the data is spread out, it feels like you're accessing files from one place.
  - **Examples**:
    - **Google File System (GFS)**: Developed by Google to handle large-scale data processing.
    - **Hadoop File System (HDFS)**: Part of the Hadoop ecosystem, designed for big data applications.
    - **AWS S3**: Amazon's cloud storage service that provides scalable storage for data.
  - **Files are**:
    - **Broken into blocks**: Large files are divided into smaller pieces called blocks to manage and store them efficiently.
    - **Blocks partitioned across machines**: These blocks are distributed across different machines to balance the load and optimize storage.
    - **Blocks often replicated**: To ensure data safety and availability, blocks are usually copied multiple times across different machines.
- *Goals*
  - **Store data not fitting on one machine**: Distributed file systems are essential for handling massive datasets that exceed the storage capacity of a single machine.
  - **Increase performance**: By distributing data, these systems can process and retrieve

data faster, improving overall performance.
- **Increase reliability/availability/fault tolerance**: Replicating data across machines ensures that even if one machine fails, the data remains accessible and safe, enhancing system reliability and fault tolerance.

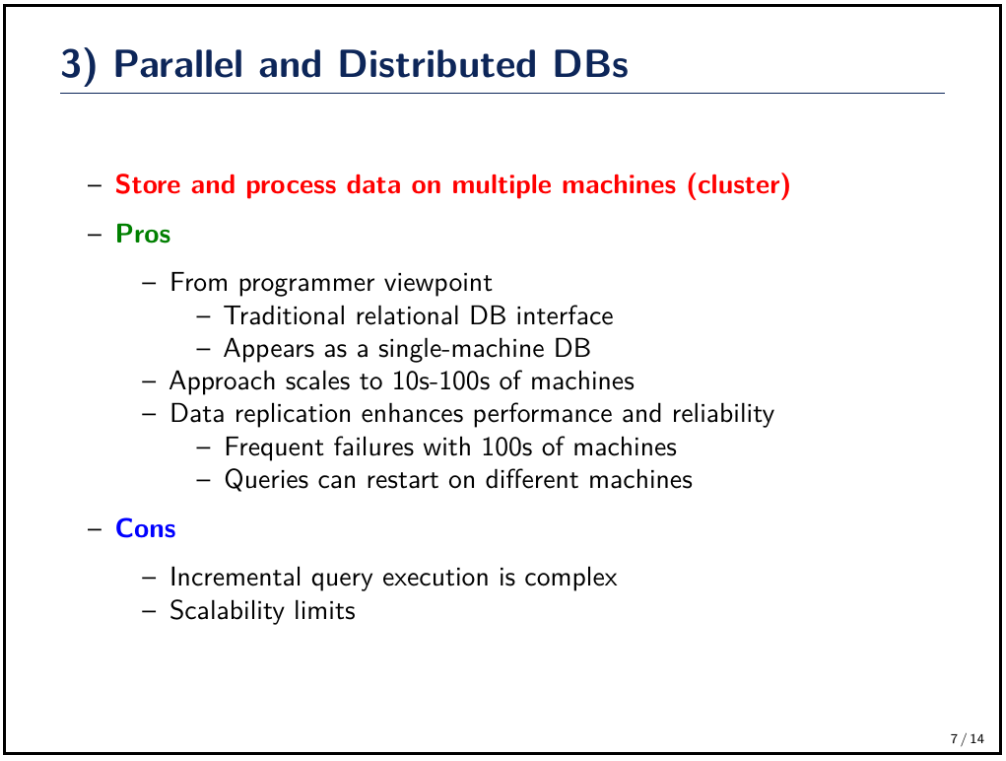
6 / 14: 2) Sharding Across Multiple DBs



- **Sharding**: This is a method used to divide and store data across multiple databases or machines. The main goal is to manage large datasets more efficiently by distributing the load.
- **Shard keys**: These are specific attributes or keys used to determine how data is partitioned. They are crucial because they dictate how data is distributed across different shards.
- **Range partition**: This involves dividing data based on a specific range of values, such as dates in a timeseries database. It's useful for organizing data that naturally falls into sequential ranges.
- **Hash partition**: This method uses a hash function to distribute data evenly across shards. It's often used to ensure a balanced distribution of data, avoiding hotspots.
- **Pros**:
  - Sharding allows a database to scale beyond the limitations of a single, centralized database. This means it can handle more users, store more data, and process requests faster.
- **Cons**:
  - To prevent data loss in case of failures, replication of data across shards is necessary,

which can be complex.
  - Maintaining data consistency across multiple shards is a significant challenge, especially when data is frequently updated.
  - Relational databases often face difficulties with enforcing constraints like foreign keys and managing transactions when data is spread across multiple machines. This can complicate database operations and integrity.

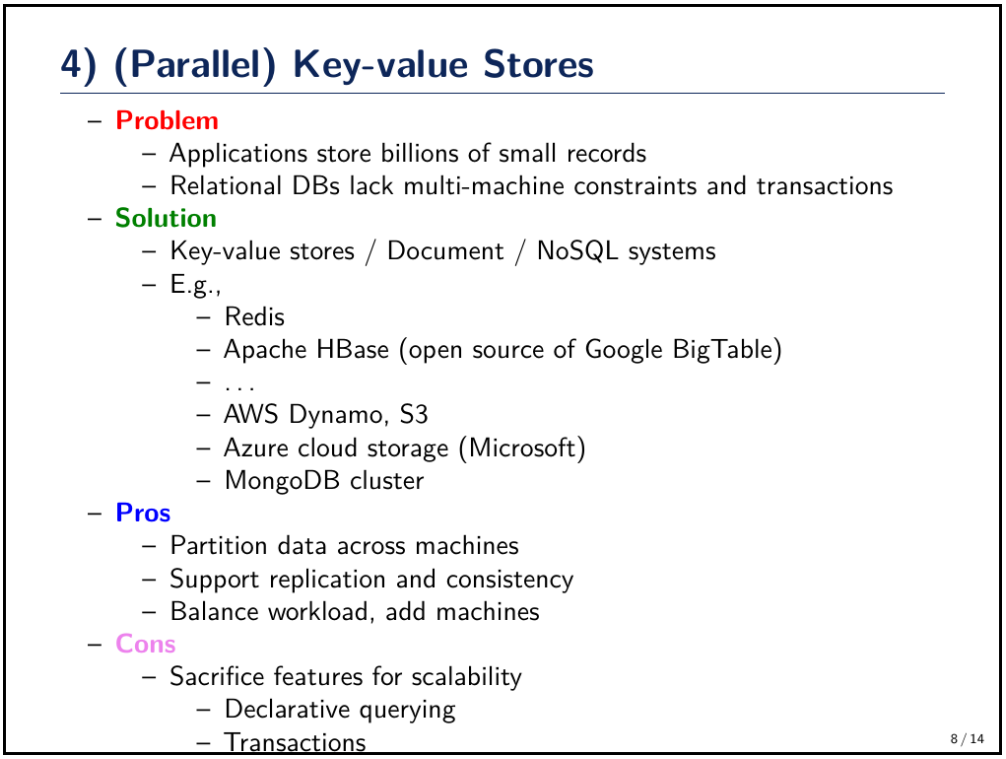
7 / 14: 3) Parallel and Distributed DBs



- **Parallel and Distributed DBs**: These databases are designed to handle large volumes of data by distributing the storage and processing tasks across multiple machines, often referred to as a *cluster*. This setup allows for more efficient data management and processing.
- **Pros**:
  - **From programmer viewpoint**: Developers can interact with these databases using a traditional relational database interface, which means they don't need to learn new tools or languages. The system is designed to appear as if it is a single-machine database, simplifying the development process.
  - **Scalability**: This approach can effectively scale to accommodate 10s to 100s of machines, allowing for significant growth in data handling capacity without a complete overhaul of the system.
  - **Data replication**: By replicating data across multiple machines, the system can enhance both performance and reliability. This is crucial because with hundreds of machines, failures are more common. If one machine fails, queries can be restarted on another machine, minimizing downtime and data loss.
- **Cons**:

  - **Incremental query execution is complex**: Managing and executing queries incrementally across multiple machines can be challenging. This complexity arises from the need to coordinate and synchronize data processing tasks across the distributed system.
  - **Scalability limits**: Although these systems can scale to a large number of machines, there are still limits to how far they can scale efficiently. Beyond a certain point, the overhead of managing the distributed system can outweigh the benefits, leading to diminishing returns in performance improvements.

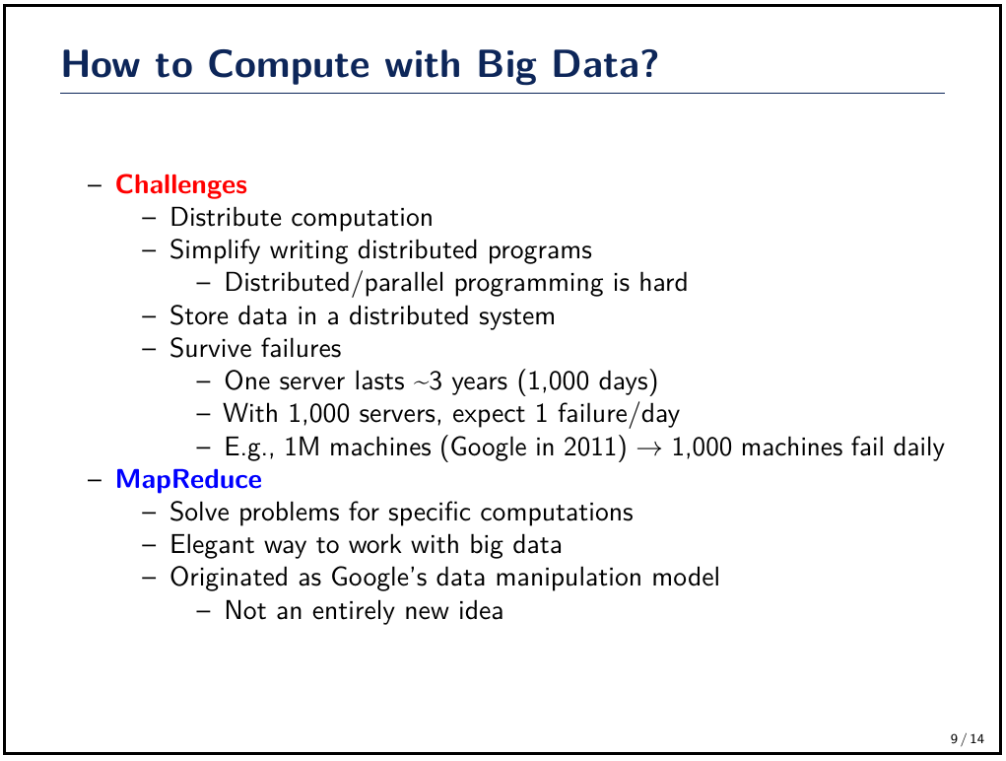
8 / 14: 4) (Parallel) Key-value Stores



– **@Problem@**
  - Many modern applications need to handle *billions of small records*. This is a huge amount of data that traditional databases struggle with.
  - **Relational databases** (like MySQL or PostgreSQL) are not designed to work efficiently across multiple machines. They also have limitations when it comes to handling transactions in a distributed environment.
– **@Solution@**
  - To address these challenges, we use **key-value stores**, which are a type of NoSQL database. These systems are designed to handle large volumes of data across many machines.
  - Examples of key-value stores include:
    - **Redis**: Known for its speed and in-memory data storage.
    - **Apache HBase**: An open-source version of Google's BigTable, designed for big data applications.
    - **AWS Dynamo and S3**: Amazon's solutions for scalable storage.
    - **Azure cloud storage**: Microsoft's cloud storage solution.

    - **MongoDB cluster**: A popular NoSQL database that supports document storage.
- **@Pros@**
  - These systems can **partition data across multiple machines**, which helps in managing large datasets efficiently.
  - They support **replication and consistency**, ensuring data is available and reliable.
  - You can **balance the workload** by adding more machines, which helps in scaling the system as needed.
- **@Cons@**
  - To achieve scalability, these systems often **sacrifice some features**:
    - They may not support **declarative querying** like SQL, which makes complex queries harder to perform.
    - **Transactions** might not be fully supported, which can be a limitation for applications needing strong consistency.
    - Retrieving data based on **non-key attributes** can be challenging, as these systems are optimized for key-based access.

9 / 14: How to Compute with Big Data?



- **How to Compute with Big Data?**
- **Challenges**
  - **Distribute computation**: When dealing with big data, it's crucial to spread the workload across multiple computers. This helps in processing large datasets efficiently.
  - **Simplify writing distributed programs**: Writing programs that run on multiple computers at once is complex. It's not easy to manage how these computers communicate and work together.

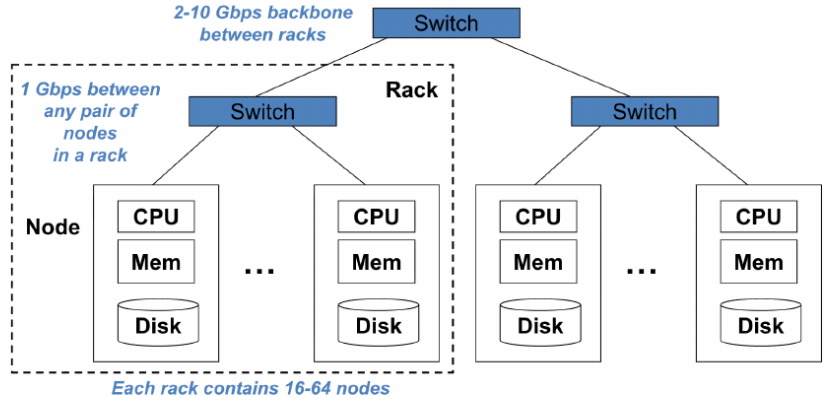
  - **Store data in a distributed system**: Big data requires storage systems that can handle vast amounts of information spread across many machines.
  - **Survive failures**: Computers can fail, and with many machines, failures are frequent. For example, a single server might last about three years. If you have 1,000 servers, you might expect one to fail every day. Companies like Google, which had around a million machines in 2011, could see about 1,000 failures daily.

- **MapReduce**
  - **Solve problems for specific computations**: MapReduce is a method designed to handle specific types of data processing tasks efficiently.
  - **Elegant way to work with big data**: It provides a straightforward approach to process large datasets by breaking down tasks into smaller, manageable parts.
  - **Originated as Google's data manipulation model**: Google developed MapReduce to handle their massive data needs. While it was innovative, the concept of dividing tasks wasn't entirely new.

10 / 14: Cluster Architecture

Cluster Architecture

- Standard architecture for big data computation
  - Cluster of commodity Linux nodes
  - Commodity network (typically Ethernet) to connect nodes
  - 2011: Google ~1M machines
  - 2025: ~10-15M (?)



10 / 14

- **Cluster Architecture**
- **Standard architecture for big data computation**
  - *Cluster of commodity Linux nodes*: This refers to using affordable, widely available hardware running Linux to build a cluster. These nodes work together to handle large-scale data processing tasks.
  - **Commodity network (typically Ethernet) to connect nodes**: Ethernet is commonly used to connect these nodes, providing a cost-effective and reliable networking

solution.
  - **2011: Google ~1M machines**: By 2011, Google had approximately one million machines in its data centers, showcasing the scale of infrastructure needed for big data processing.
  - **2025: ~10-15M (?)**: This projection suggests a significant increase in the number of machines, reflecting the growing demand for data processing capabilities.

- **Diagram Explanation**
  - The diagram illustrates a typical cluster setup:
    - **Node**: Each node contains a CPU, memory, and disk storage, forming the basic unit of computation and storage.
    - **Rack**: A collection of nodes, typically 16-64, connected by a switch. Nodes within a rack communicate at 1 Gbps.
    - **Switches**: Connect racks with a backbone network, providing 2-10 Gbps between racks, facilitating efficient data transfer across the cluster.

11 / 14: Cluster Architecture

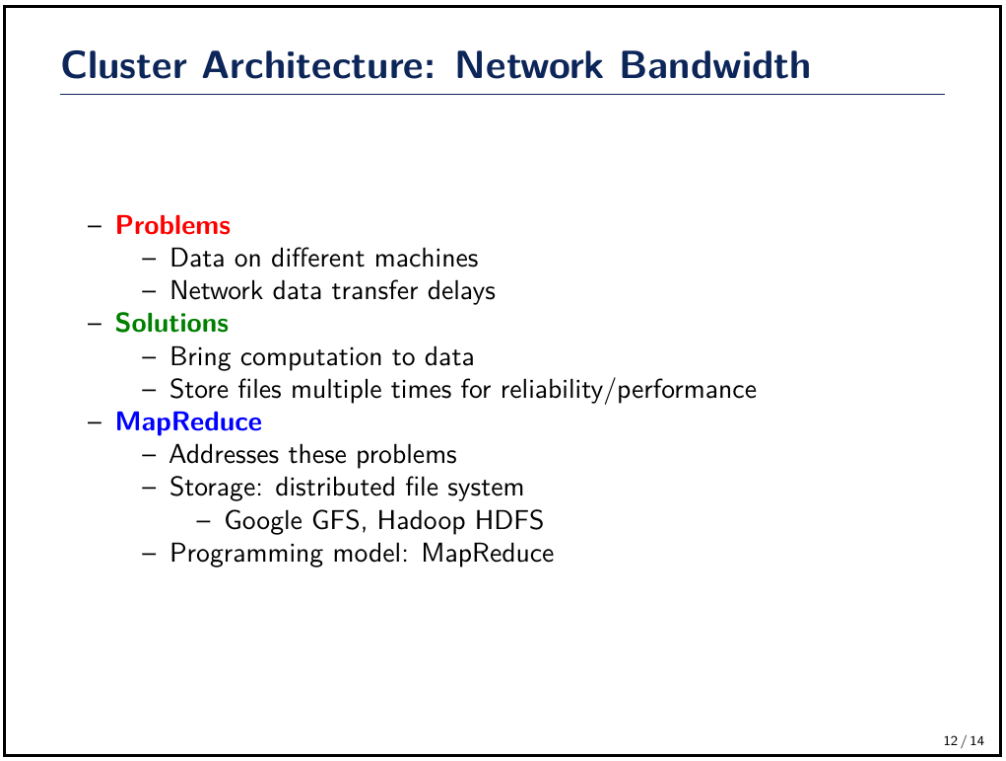
Cluster Architecture

11 / 14

- **Cluster Architecture**
  - *Definition*: A cluster architecture refers to a group of interconnected computers that work together as a single system to provide high availability, scalability, and performance.
  - **Components**:
    - *Nodes*: Each computer in the cluster is called a node. Nodes can be physical machines or virtual instances.
    - *Networking*: Nodes are connected through a network, allowing them to communicate and share resources.

  - *Storage*: Shared or distributed storage systems are often used to ensure data availability and redundancy.
- **Benefits**:
  - *Scalability*: Easily add more nodes to increase computing power.
  - *Fault Tolerance*: If one node fails, others can take over its tasks, minimizing downtime.
  - *Load Balancing*: Distributes workloads evenly across nodes to optimize resource use.
- **Use Cases**:
  - Commonly used in data centers, cloud computing, and high-performance computing environments.
  - Ideal for applications requiring large-scale data processing, such as big data analytics and scientific simulations.

12 / 14: Cluster Architecture: Network Bandwidth



- **Cluster Architecture: Network Bandwidth**
- **@Problems@**
  - **Data on different machines**: In a cluster, data is often spread across multiple machines. This distribution can lead to inefficiencies because accessing data from different machines can be slow and cumbersome.
  - **Network data transfer delays**: When data needs to be transferred over a network, it can introduce delays. This is because network bandwidth is limited, and transferring large amounts of data can take significant time, slowing down processing.
- **@Solutions@**

  - **Bring computation to data**: Instead of moving data to where the computation happens, it's more efficient to move the computation to where the data resides. This reduces the need for data transfer and speeds up processing.
  - **Store files multiple times for reliability/performance**: By storing copies of files on different machines, systems can ensure that data is available even if one machine fails. This redundancy also allows for faster data access, as the system can retrieve data from the nearest copy.

- **@MapReduce@**

  - **Addresses these problems**: MapReduce is a programming model that helps solve the issues of data distribution and network delays.
  - **Storage: distributed file system**: Systems like Google GFS and Hadoop HDFS are designed to store data across multiple machines efficiently. They manage data distribution and replication automatically.
  - **Programming model: MapReduce**: This model simplifies processing large data sets by breaking down tasks into smaller, manageable chunks that can be processed in parallel, reducing the need for extensive data transfer.

13 / 14: Storage Infrastructure

Storage Infrastructure

- **Problem**
  - Store data persistently and efficiently despite node failures
- **Typical data usage pattern**
  - Huge files (100s of GB to 1TB)
  - Common operations: reads and appends
  - Rare in-place updates
- **Solution**
  - Distributed file system
  - Store files across multiple machines
  - Files are:
    - Broken into blocks
    - Partitioned across machines
    - Replicated across machines
  - Provide a single file-system view to clients

13 / 14

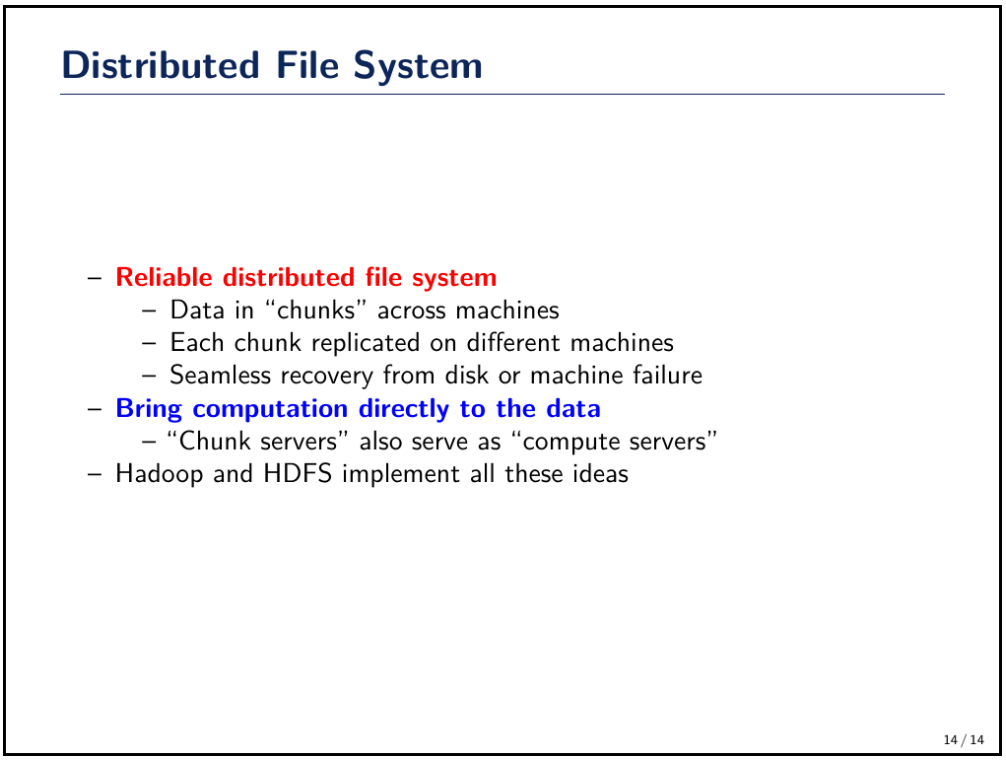
- **Storage Infrastructure**

- **@Problem@**

  - The main challenge here is to ensure that data is stored in a way that it remains accessible and intact even if some of the storage nodes (computers) fail. This is crucial because in large systems, hardware failures are common, and losing data can be very costly.

- **@Typical data usage pattern@**
  - In big data environments, files are often very large, ranging from hundreds of gigabytes to a terabyte. This means that traditional storage solutions might not be efficient or feasible.
  - The most common operations performed on these files are reading data and adding new data to the end of files (appending). This is because data is often collected and analyzed in bulk.
  - Updating data in place (changing existing data) is rare. This is because it can be complex and time-consuming to modify large files directly.
- **@Solution@**
  - A distributed file system is used to tackle these challenges. This system spreads data across multiple machines, which helps in managing large files and ensuring data availability.
  - Files are divided into smaller pieces called blocks. These blocks are then distributed across different machines. This distribution helps in balancing the load and improving access speed.
  - To protect against data loss, each block is replicated, meaning copies are stored on multiple machines. This way, if one machine fails, the data can still be accessed from another machine.
  - Despite the data being spread out, the system provides a unified view to the users, making it appear as if all the data is stored in one place. This simplifies data management and access for users.

14 / 14: Distributed File System



- **Distributed File System**
- *Reliable distributed file system*
  - **Data in "chunks" across machines**: In a distributed file system, data is broken down into smaller pieces called "chunks." These chunks are stored across multiple machines in a network. This approach helps in managing large datasets efficiently and ensures that the system can handle large volumes of data without any single machine becoming a bottleneck.
  - **Each chunk replicated on different machines**: To ensure data reliability and availability, each chunk is copied and stored on multiple machines. This replication means that if one machine fails, the data is still accessible from another machine, minimizing the risk of data loss.
  - **Seamless recovery from disk or machine failure**: The system is designed to automatically recover from failures. If a disk or machine fails, the system can quickly switch to using the replicated data from other machines, ensuring continuous operation without significant downtime.
- *Bring computation directly to the data*
  - **"Chunk servers" also serve as "compute servers"**: In this model, the servers that store the data chunks also perform computations on the data. This reduces the need to move large amounts of data across the network, which can be time-consuming and resource-intensive. By processing data where it is stored, the system becomes more efficient and faster.
- **Hadoop and HDFS implement all these ideas**: Hadoop is a popular framework that uses the Hadoop Distributed File System (HDFS) to implement these concepts. HDFS is designed to store and manage large datasets across distributed systems, making it a key component in big data processing. Hadoop leverages these principles to provide a scalable and reliable platform for data storage and processing.